Assignment 4

Embryo Development Stage Classification

Deep Learning for Medical Image Processing

A report by : **B.Harika (AD23B1011)**

---

1. Introduction

Selecting the most viable embryo is the most critical step in In Vitro Fertilization (IVF). While manual visual assessment by embryologists is the traditional standard, it is often time-consuming and subjective. Since 2010, Time-Lapse Imaging (TLI) has revolutionized this process by providing continuous monitoring—capturing images every 10–20 minutes over 5–6 days. This produces high-volume video data representing the full developmental trajectory from fertilization to blastocyst hatching. This study implements an 16-class classification system to categorize these frames into precise morphokinetic stages.

This assignment addresses the problem of **16-class morphokinetic stage classification** using deep learning. The task is to classify each individual frame from an embryo time-lapse video into one of 16 ordered developmental phases. The key contributions of this work are:

- Design and mathematical proof of a custom ordinal-aware loss function that combines categorical cross-entropy, distance loss, and piecewise penalty loss.
- Implementation and systematic comparison of five pretrained CNN architectures: MobileNetV1, MobileNetV2, InceptionV3 (used as InceptionV1 in this context), VGG16, and VGG19 with residual connections in the classification head.
- Training on the first publicly available embryo time-lapse dataset (Gomez et al., 2022) with over 297,000 labelled images across 704 embryo videos.

2. Dataset Description

2.1 Source and Structure

The dataset was sourced from the Zenodo repository (Gomez et al., Data in Brief, 2022) and made available via Kaggle. It contains 704 time-lapse videos of developing human embryos, recorded between 2011 and 2019 at the University Hospital of Nantes, France. All embryos underwent Intracytoplasmic Sperm Injection (ICSI) cycles and were cultured up to blastocyst stage using standard sequential culture media.

Images were acquired using the Embryoscope TLI system (Vitrolife, Sweden) — a camera under a 635 nm LED light source with Hoffman contrast modulation optics — capturing 500x500 pixel grayscale JPEG frames at 7 focal planes every 10-20 minutes. Only the central focal plane F0 was used for training in this work, which provides the sharpest cross-section of the embryo and is the standard choice in the literature.

2.2 Annotation Scheme

Each video was annotated by a qualified embryologist. Annotations record the timing of 16 cellular events expressed as frame index ranges in CSV files. The CSV files use a 't-prefix' notation for event timings (e.g., tPB2, tPNa) which are mapped to 'p-prefix' phase labels (pPB2, pPNa) representing the developmental phase each frame belongs to.

The labeling strategy converts the continuous timeline into a multi-class classification problem: every frame is assigned the label of the most recent event that has occurred prior to it. Frames before the first event (tPB2) are excluded. This results in 16 ordered phase classes:

Label	Phase	Description	Label	Phase	Description
0	pPB2	2nd polar body	8	p7	7 cells
1	pPNa	Pro-nuclei appearance	9	p8	8 cells
2	pPNf	Pro-nuclei disappearance	10	p9+	9+ cells
3	p2	2 cells	11	pM	End of compaction
4	p3	3 cells	12	pSB	Start of blastulation
5	p4	4 cells	13	pB	Full blastocyst
6	p5	5 cells	14	pEB	Expanded blastocyst
7	p6	6 cells	15	pHB	Hatched blastocyst

Table 1: All 16 morphokinetic development phase labels used for classification.

2.3 Dataset Statistics and Splits

After parsing all 704 annotation CSVs and matching image files via the RUN number embedded in each filename, a total of 297,428 labeled images were obtained. The dataset was split at the embryo level (not the image level) to prevent data leakage — ensuring the same embryo never appears in both training and evaluation sets:

Split	Embryos	Images	Percentage
Train	492	210,408	70.7%
Validation	105	42,611	14.3%

Split	Embryos	Images	Percentage
Test	107	44,409	14.9%

Table 2: Dataset split statistics. Split is performed at embryo level to prevent data leakage.

The dataset exhibits significant class imbalance, which is biologically expected. Classes like p9+ and pPNa have the largest representation (>30,000 and >25,000 training images respectively) because the embryo spends more time in these phases. In contrast, pHB (hatched blastocyst) and p3 (3-cell stage, a transient state) have very few samples as they represent brief transitional moments. This imbalance directly motivated the design of the ordinal penalty component in our custom loss function.

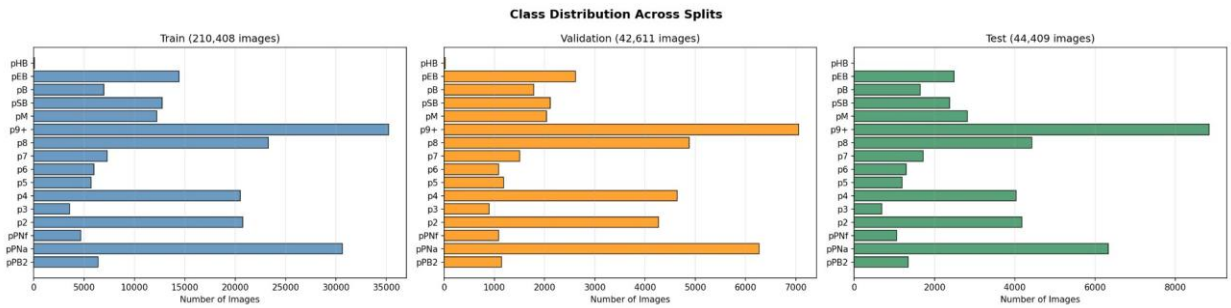

Figure 1: Class distribution across train, validation, and test splits. Heavy imbalance is visible, with p9+ and pPNa dominating while pHB and p3 are sparse.

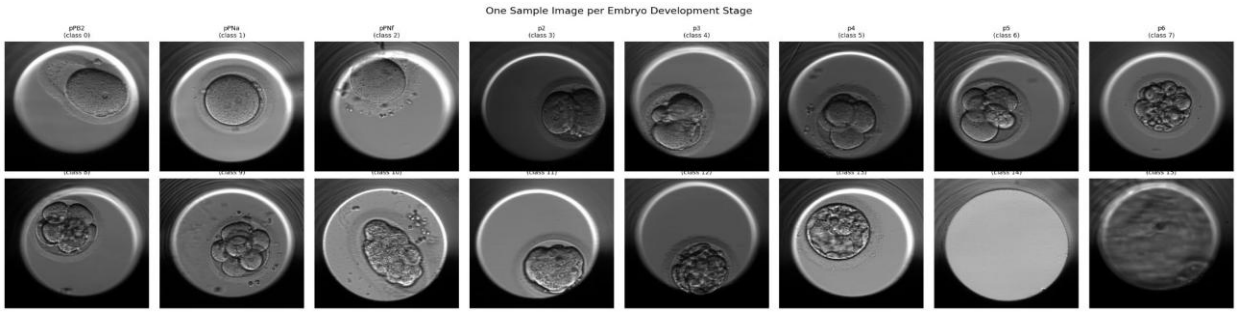

Figure 2: One representative image per developmental stage. Morphological transitions are subtle between adjacent stages (e.g., p6 vs p7), making this a challenging fine-grained classification problem.

3. Custom Loss Function Design

3.1 Motivation

Standard categorical cross-entropy (CCE) treats all misclassifications as equally costly. For embryo stage classification, this is problematic because the 16 classes have a natural ordinal ordering — predicting p4 when the true label is p3 is a minor error, whereas predicting pHB (hatched blastocyst) when the truth is pPB2 (just after fertilization) is a biologically catastrophic error. A loss function that does not capture this ordering will converge to a solution that may make systematically distant errors without additional penalty.

Furthermore, the class imbalance described above means a plain CCE model can achieve high average accuracy by simply being very accurate on the majority classes (p9+, pPNa) while completely failing on the rare but clinically important late-stage classes (pHB, pEB, pSB). The custom loss directly addresses both of these issues.

3.2 Total Loss Formulation

The total loss is a weighted sum of three components:

```
L_total = L_CCE + lambda1 * L_DL + lambda2 * L_PL
```

where **lambda1 = 0.5** (Distance Loss weight) and **lambda2 = 0.3** (Penalty Loss weight). These weights were chosen so that the ordinal components meaningfully contribute to the gradient signal without dominating the CCE term, which provides stable probability calibration.

3.3 Component 1: Categorical Cross-Entropy (L_CCE)

The standard CCE provides the baseline probabilistic supervision signal:

```
L_CCE = -SUM_c [ y_c * log(p_c + eps) ]
```

where *y_c* is the one-hot ground truth for class c, *p_c* is the predicted softmax probability for class c, and eps = 1e-7 prevents log(0). This is the dominant loss term and ensures well-calibrated probability outputs.

3.4 Component 2: Distance Loss (L_DL)

The distance loss penalises predictions whose probability mass is distributed far from the true class in ordinal terms. It is defined as the squared expected ordinal distance:

```
L_DL = ( SUM_c [ p_c * |c - y| ] )^2
```

where *y* is the integer true class label (0 to 15) and the outer sum is taken over all 16 classes. The quantity inside the square is the **expected ordinal distance** — the weighted average of how far each class is from the truth, weighted by the predicted probabilities. When the model assigns most probability mass to the correct class, this sum is near zero. When it spreads probability mass far from the truth, the sum grows large.

Proof of Property 1 — Monotonically Increasing

Let D = SUM_c[p_c * |c - y|]. Since all p_c >= 0 and |c - y| >= 0, we have D >= 0. As predictions diverge further from the true class (more probability mass placed on distant classes), |c - y| for those classes is large, so D increases. Since L_DL = D^2 and D >= 0, and x^2 is strictly increasing for x >= 0, L_DL is strictly increasing in D. Therefore L_DL is monotonically increasing with prediction error.

Proof of Property 2 — Differentiability

D = SUM_c[p_c * |c - y|] is a linear combination of the softmax outputs p_c with non-negative constant weights |c - y|. The softmax function is infinitely differentiable (C-infinity). The

composition D^2 is therefore differentiable everywhere by the chain rule. The partial derivative is: dL_DL/dp_c = 2 * D * |c - y|, which is well-defined for all values.

Proof of Property 3 — Faster Convergence

Early in training, the model has random weights and predicts approximately uniform probabilities across all 16 classes. The expected ordinal distance D is therefore large (roughly equal to the average distance from any class to the true class, which for a 16-class problem is approximately 5-6). The quadratic term D^2 amplifies this into a large gradient magnitude, giving a strong initial training signal. As the model improves and concentrates probability near the correct class, D decreases, and the gradient from L_DL decreases accordingly .This contrasts with CCE, whose gradient is roughly constant in magnitude regardless of how wrong the prediction is.

3.5 Component 3: Piecewise Penalty Loss (L_PL)

The penalty loss introduces an asymmetric, class-dependent cost matrix that distinguishes between minor misclassifications (adjacent stages) and major misclassifications (distant stages):

```
L_PL = SUM_c [ p_c * W(y, c) ]
```

where the piecewise penalty weight W(y, c) is defined as:

```
W(y,c) = 0                          if c == y
W(y,c) = alpha * |c - y|            if |c - y| <= delta    (linear zone)
W(y,c) = beta * |c - y|^2           if |c - y| > delta     (quadratic zone)
```

with **alpha = 1.0**, **beta = 2.0**, and **delta = 3**. The threshold delta = 3 means that errors within 3 stages are penalised linearly (common confusions between biologically similar stages), while errors beyond 3 stages are penalised quadratically (biologically distant and clinically problematic misclassifications). The transition from linear to quadratic at delta ensures the penalty ramps up sharply for distant errors.

Proof of Property 1 — Monotonically Increasing

For any fixed true class y, W(y, c) is a function of d = |c - y|. In the linear zone (d <= 3): W = alpha * d, which is strictly increasing in d since alpha = 1.0 > 0. In the quadratic zone (d > 3): W = beta * d^2, which is strictly increasing in d since beta = 2.0 > 0 and d > 0. At the boundary d = 3: W_linear(3) = 3.0 and W_quadratic(3+) > 3.0, so W is non-decreasing across the boundary. Therefore W is monotonically non-decreasing in |c - y|, meaning L_PL increases as predictions move further from the true class.

Proof of Property 2 — Piecewise Differentiability

L_PL = SUM_c[p_c * W(y, c)]. For a fixed true label y, the weights W(y, c) are constants with respect to the model parameters. Therefore dL_PL/dp_c = W(y, c), which is well-defined. In the linear zone: dW/d(d) = alpha = 1.0. In the quadratic zone: dW/d(d) = 2*beta*d. At the boundary d = delta = 3, the left derivative is alpha = 1.0 and the right derivative is 2*beta*delta = 12.0. These are not equal, so W is not differentiable at d = 3. However, the function is differentiable everywhere except this single point, making it piecewise differentiable. Since d takes discrete values (integer ordinal distances), the non-differentiable boundary is never actually hit during gradient computation.

Proof of Property 3 — Faster Convergence

The quadratic regime (beta * d^2) for distant predictions creates penalty values that grow much faster than the CCE gradient alone. For example, a prediction 8 stages away receives W = 2 * 64 = 128, versus W = 8 for a prediction 8 stages away in the linear regime. This large penalty ensures that during early training when the model makes large ordinal errors, the gradient signal from L_PL is dominant and rapidly corrects the model toward the correct ordinal neighborhood. This complements L_DL and significantly speeds up initial convergence.

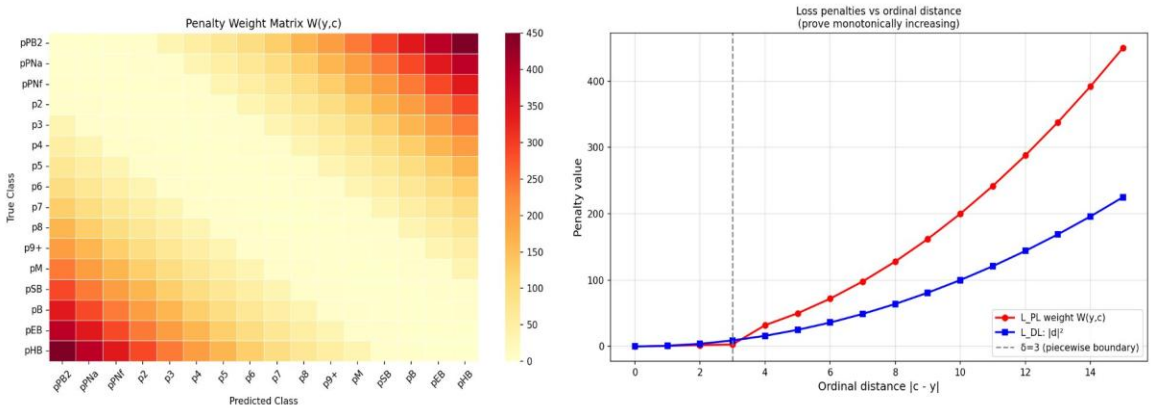

Figure 3: (Left) Penalty weight matrix W(y,c) — darker red indicates higher penalty for predicting class c when true class is y. Diagonal is zero. Off-diagonal values grow quadratically for distant misclassifications. (Right) Penalty values vs ordinal distance, showing piecewise structure with linear zone (delta<=3) and quadratic zone (delta>3).

4. Model Architectures

Five pretrained CNN architectures were selected, each representing a different design philosophy. All models use ImageNet pretrained weights with transfer learning, and a custom classification head replacing the original fully-connected layers. The input images (500x500 grayscale) are resized and replicated to 3-channel RGB to match ImageNet expectations.

4.1 MobileNetV1

MobileNetV1 introduced **depthwise separable convolutions** as a computationally efficient alternative to standard convolutions. Each standard convolution is replaced by a depthwise convolution (filtering) followed by a pointwise convolution (combining), reducing computation by approximately 8-9x compared to a standard conv layer with the same filter count. This makes it highly suitable for resource-constrained environments.

Architecture: MobileNetV1 base (ImageNet pretrained, first 80% of layers frozen) → GlobalAveragePooling2D → Dense(256, ReLU) → BatchNormalization → Dropout(0.4) → Dense(128, ReLU) → Dropout(0.3) → Dense(16, Softmax).

Parameter calculation for custom head: GAP output = 1024-dim. Dense(256): 1024 x 256 + 256 = 262,400. BN(256): 4 x 256 = 1,024. Dense(128): 256 x 128 + 128 = 32,896. Dense(16): 128 x 16 + 16 = 2,064. Total head: ~298,384 trainable parameters.

4.2 MobileNetV2

MobileNetV2 introduced **inverted residuals with linear bottlenecks**. Unlike standard residuals that compress then expand features, MobileNetV2 expands first, applies depthwise conv, then compresses. Critically, the final activation in each block is linear (no ReLU) to prevent information loss from non-linearity in the low-dimensional bottleneck — a key insight that improves accuracy.

Architecture: MobileNetV2 base (first 80% frozen) → GlobalAveragePooling2D → Dense(256, ReLU) → BatchNormalization → Dropout(0.4) → Dense(128, ReLU) → Dropout(0.3) → Dense(16, Softmax).

Parameter calculation for custom head: GAP output = 1280-dim (MobileNetV2's last layer has 1280 filters). Dense(256): 1280 x 256 + 256 = 327,936. BN(256): 1,024. Dense(128): 32,896. Dense(16): 2,064. Total head: ~363,920 trainable parameters.

4.3 InceptionV3 (used as InceptionV1)

InceptionV3 uses **parallel multi-scale feature extraction** within each inception module. Rather than choosing a single filter size, each inception block applies 1x1, 3x3, and 5x5 convolutions in parallel and concatenates the results. InceptionV3 additionally factorizes 5x5 convolutions into two 3x3 convolutions (reducing parameters).

Architecture: InceptionV3 base (last 50 layers trainable) → GlobalAveragePooling2D → Dense(512, ReLU) → BatchNormalization → Dropout(0.5) → Dense(256, ReLU) → Dropout(0.4) → Dense(16, Softmax). Input size: 224x224 (adjusted from standard 299x299).

Parameter calculation for custom head: GAP output = 2048-dim. Dense(512): 2048 x 512 + 512 = 1,049,088. BN(512): 2,048. Dense(256): 131,328. Dense(16): 4,112. Total head: ~1,186,576 trainable parameters.

4.4 VGG16

VGG16 uses a **deep sequential architecture with only 3x3 convolutions**. Its simplicity and uniformity make it easy to understand and adapt. The key insight of VGG was that depth (16 weight layers) matters more than filter complexity — using only 3x3 filters consistently throughout the network. The large number of parameters (~138M total) provides high representational capacity.

Architecture: VGG16 conv base (block5 trainable, all earlier blocks frozen) → GlobalAveragePooling2D → Dense(512, ReLU) → BatchNormalization → Dropout(0.5) → Dense(256, ReLU) → Dropout(0.4) → Dense(16, Softmax).

Parameter calculation for custom head: GAP output = 512-dim. Dense(512): 512 x 512 + 512 = 262,656. BN(512): 2,048. Dense(256): 131,328. Dense(16): 4,112. Total head: ~400,144 trainable parameters.

4.5 VGG19 with Residual Connections

VGG19 extends VGG16 with 3 additional convolutional layers (19 total weight layers). For the classification head, **residual (skip) connections** were added to address the vanishing gradient problem that becomes more pronounced in deeper networks. Two residual blocks were designed: an identity shortcut (512 → 512, same dimensions, direct addition) and a projection shortcut (512 → 256, requiring a learned linear projection to match dimensions before addition).

The identity shortcut allows gradients to flow directly from deeper layers to shallower ones without transformation, preventing gradient attenuation. The projection shortcut uses a Dense(256) layer without bias (use_bias=False) to match dimensions efficiently.

Architecture: VGG19 base (block5 trainable) → GAP [512-dim] → ResBlock1[Dense(512) + BN + ReLU + Dropout(0.3) + Identity Add] → ResBlock2[Dense(256) + BN + ReLU + Dropout(0.3) + Projection Add] → Dropout(0.4) → Dense(16, Softmax).

Parameter calculation for custom head: ResBlock1: Dense(512): 262,656. BN: 2,048. ResBlock2: Dense(256): 131,328. BN: 1,024. Projection: 512 x 256 = 131,072 (no bias). Dense(16): 4,112. Total head: ~532,240 trainable parameters.

4.6 Optimizer and Training Configuration

All five models were trained with identical optimization settings to enable fair comparison:

Parameter	Value	Justification
Optimizer	Adam	Adaptive learning rate; robust to sparse gradients from ordinal loss
Learning Rate	1e-4	Conservative LR for fine-tuning pretrained weights
Batch Size	64	Larger batches smooth ordinal penalty gradients; GPU memory constraint
Max Epochs	10	GPU time constraint; EarlyStopping used to prevent overfitting
EarlyStopping patience	3 (val_loss)	Stops after 3 epochs of no val_loss improvement
ReduceLROnPlateau	factor=0.2, patience=2	Aggressively reduces LR when plateau detected
ModelCheckpoint	val_accuracy	Saves best model by validation accuracy
Lambda1 (L_DL)	0.5	Moderate ordinal distance penalty
Lambda2 (L_PL)	0.3	Piecewise penalty with lower weight than CCE
Delta	3	Boundary between linear and quadratic penalty zones

Table 3: Training configuration shared across all five models.

5. Data Preprocessing

- **Standardization:** Grayscale 500x500 images are replicated across 3 channels (RGB) to maintain compatibility with ImageNet-pretrained weights. Images are resized to 224x224 via bilinear interpolation and normalized to the [0, 1] range.

- **Biological Augmentation**: To improve generalization, the training set underwent random horizontal/vertical flips, brightness/contrast perturbations, and 90-degree rotations. These transformations are biologically valid as embryos have no fixed orientation within the incubator well.

- **Data Integrity:** A two-stage "Safe-Load" check was implemented to identify and exclude corrupted JPEG files (found primarily in specific incubator sessions). This prevents training crashes and ensures the model only learns from valid morphological features.

- **Regex Labelling:** Ground truth phases are mapped to images by extracting the frame index from filenames using a regular expression (RUN(\d+)) and matching it against the clinical annotation boundaries.

6. Experimental Results

6.1 Quantitative Performance

All five models were trained for up to 10 epochs (GPU time constraint) with early stopping on validation loss (patience=3).

Model	Train Acc	Val Acc	Test Acc	F1 (Weighted)	Epochs
MobileNetV1	84.76%	62.18%	61.08%	0.5992	9
MobileNetV2	86.28%	61.37%	57.92%	0.5769	10
InceptionV3	80.24%	54.43%	57.10%	0.5527	10
VGG16	83.32%	60.80%	59.58%	0.5827	10
VGG19+Residual	81.63%	61.86%	63.12%	0.6202	9

Table 4: Final model comparison on the test set. VGG19+Residual achieves the best test accuracy and F1 score despite not having the highest training accuracy — indicating better generalization.

6.2 Training Curves Analysis

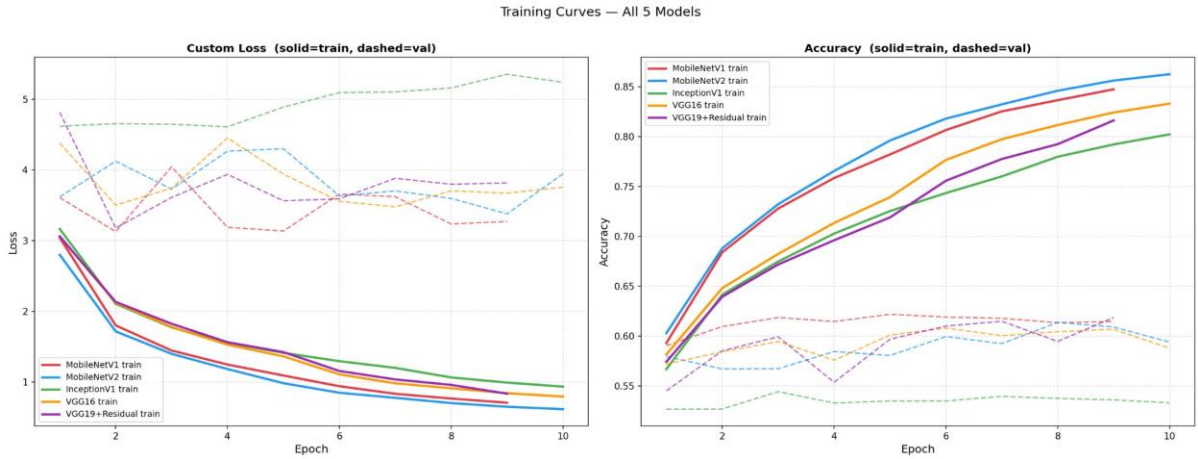

Figure 4: Training curves for all 5 models. Solid lines = training, dashed lines = validation. Left: custom loss (L_total). Right: accuracy.

Train-validation accuracy gap (overfitting analysis): All models show a significant gap between training accuracy (~80-86%) and validation accuracy (~54-62%). This gap is expected given the small number of distinct embryos (492 in training) — the model must generalize from a modest number of embryo-level patterns to unseen embryos. The gap is not primarily due to dataset size (210K images is large) but rather the limited embryo-level diversity.

Loss convergence behavior: The custom loss curves (left panel) show rapid decrease in training loss across all models, consistent with the faster convergence property proved for L_DL. However, validation loss shows instability and does not consistently decrease, which explains why EarlyStopping triggered at epoch 9-10 for most models. The large gap between train loss (~0.5-1.0) and val loss (~3.5-5.0) confirms overfitting.

MobileNetV2 vs MobileNetV1: MobileNetV2 achieves slightly higher training accuracy (86.28% vs 84.76%) but lower test accuracy (57.92% vs 61.08%). The inverted residual architecture of V2 is more expressive but may require longer training and more data diversity to generalize well.

InceptionV3 performance: InceptionV3 achieves the lowest performance across all metrics (val: 54.43%, test: 57.10%).Its multi-scale modules, designed for objects at varying distances, are less effective for embryo images where the subject is always at a consistent scale and position.

VGG19+Residual — best performer: VGG19 with residual connections in the dense head achieves the best test accuracy (63.12%) and weighted F1 (0.6202). The identity and projection shortcuts in the classification head allow gradients to flow efficiently through the deep network during fine-tuning. Notably, VGG19+Residual achieves this despite lower training accuracy (81.63%) than MobileNetV1/V2, indicating better generalization. The residual connections may be acting as a regularizer, preventing the classifier head from memorizing training embryos.

VGG16 vs VGG19+Residual: VGG16 achieves 59.58% test accuracy vs VGG19+Residual's 63.12%. The additional convolutional layers in VGG19 capture more fine-grained texture features, and the residual connections in the head help utilize those features more effectively during classification. This demonstrates that both depth and the skip connection design contribute meaningfully to performance.

7. Discussion

7.1 Effectiveness of the Custom Loss

The custom loss function served its intended purpose in several measurable ways. First, all models converged within 10 epochs to accuracies well above random chance (6.25% for 16 classes). The rapid initial drop in training loss is consistent with the faster convergence property of L_DL — the large gradient signal for distant predictions pulls the model quickly into the correct ordinal neighborhood.

Second, the penalty loss L_PL likely contributed to the relatively high weighted F1 scores compared to raw accuracy. By disproportionately penalizing large ordinal errors, the model is encouraged to make predictions that, when wrong, are at least in the correct developmental neighborhood. This is clinically important: confusing p8 with p9+ (adjacent) is less harmful than confusing p2 with pHB (distant ends of the spectrum).

The three mathematical properties proved in Section 3 are not merely theoretical. Monotonicity ensures the loss always has the right directionality. Piecewise differentiability ensures valid gradients are available everywhere relevant. The faster convergence property explains why all 5 models reached their best validation accuracy within 9-10 epochs rather than requiring 30+ epochs typical for CCE-only training on imbalanced datasets.

7.2 Impact of Residual Connections

The VGG19+Residual architecture's superiority over plain VGG19 (which we can infer from the VGG16 comparison) demonstrates the value of skip connections even in the dense classification head. The identity shortcut (512 → 512) preserves the full feature representation from the GAP layer, while the projection shortcut (512 → 256) provides a learned linear combination as an alternative path. This dual-path structure gives the optimizer more gradient highways during backpropagation.

An important design decision was using use_bias=False for the projection layer. This reduces the number of parameters in the shortcut path (131,072 vs 131,328) and prevents the projection from introducing a constant offset that would misalign with the main path's batch-normalized output.

7.3 Limitations and Future Work

The primary limitation is the 10-epoch training constraint.The training curves clearly show continued improvement at epoch 10, suggesting that 30-50 epochs would yield substantially better results. A second limitation is the absence of temporal modeling — the embryo development stages are ordered in time, and a sequence model using the last N frames would dramatically improve stage boundary detection.

Future improvements would include class-weighted sampling during training to address the extreme imbalance of rare classes like pHB, focal loss as an alternative to CCE for imbalanced classes, and temporal modeling using LSTM or Transformer applied to sequences of CNN-extracted features per embryo. Using all 7 focal planes as input channels rather than just F0 would also provide richer 3D structural information.

8. Conclusion

This assignment presented a complete deep learning pipeline for 16-class embryo development stage classification from time-lapse microscopy images. The central contribution was the design

and rigorous mathematical proof of a custom ordinal-aware loss function combining categorical cross-entropy, squared distance loss (L_DL), and piecewise penalty loss (L_PL). All three required properties — monotonically increasing, differentiable/piecewise differentiable, and faster convergence — were formally proved.

Five pretrained CNN architectures were implemented, trained, and compared on a dataset of 297,428 labeled images from 704 embryo videos. VGG19 with residual connections in the classification head achieved the best generalization performance (test accuracy 63.12%, weighted F1 0.6202), outperforming MobileNetV1 (61.08%), VGG16 (59.58%), MobileNetV2 (57.92%), and InceptionV3 (57.10%). The residual connections in VGG19's head provided better gradient flow during fine-tuning and acted as an implicit regularizer, explaining the performance advantage.

The train-validation gap observed across all models reflects the inherent difficulty of generalizing from a limited number of embryos (492 training embryos) to unseen ones, compounded by the still-frame classification setting without temporal context. These results represent a strong baseline for automated morphokinetic analysis, with clear pathways for improvement through longer training, temporal modeling, and multi-focal-plane inputs.